Aim

To design and simulate an Automotive Anti-lock Braking System (ABS) using Embedded C with interrupt handling and real-time constraints. The system monitors wheel speed, detects wheel lock conditions during braking, and activates the ABS mechanism to prevent wheel locking and improve vehicle stability.

Problem Description

An Anti-lock Braking System (ABS) is an important automotive safety feature that prevents vehicle wheels from locking during sudden or hard braking. Wheel locking can cause skidding, reduce steering control, and increase stopping distance, especially on slippery roads.

In this project, an Embedded C simulation of an Automotive ABS system is developed. The system continuously monitors wheel speed using a simulated sensor. When the brake is applied, an interrupt is generated to ensure an immediate system response. If the wheel speed falls below a predefined threshold indicating wheel lock, the controller activates the ABS by reducing brake pressure, allowing the wheel to rotate again. The system operates under real-time constraints to ensure quick and reliable braking decisions.

Objectives

- Design an Embedded C simulation of an Automotive ABS system.
- Implement interrupt handling for brake activation.
- Simulate real-time monitoring of wheel speed.
- Detect wheel lock conditions using sensor data.
- Activate the ABS mechanism when wheel lock is detected.
- Simulate brake pressure control to prevent skidding.
- Verify system performance using multiple test cases.

Functional Requirements

Inputs

- Brake switch input.
- Wheel speed sensor input (simulated).
- Vehicle speed data.

Processing

- Continuously monitor wheel speed.
- Detect brake activation using interrupts.
- Compare wheel speed with a predefined threshold.
- Determine whether wheel lock has occurred.
- Control the ABS operation based on wheel speed.

Outputs

- ABS status indication.
- Brake pressure control signal (simulated).
- Status messages displayed on the Serial Monitor.

System Requirements

Hardware Requirements

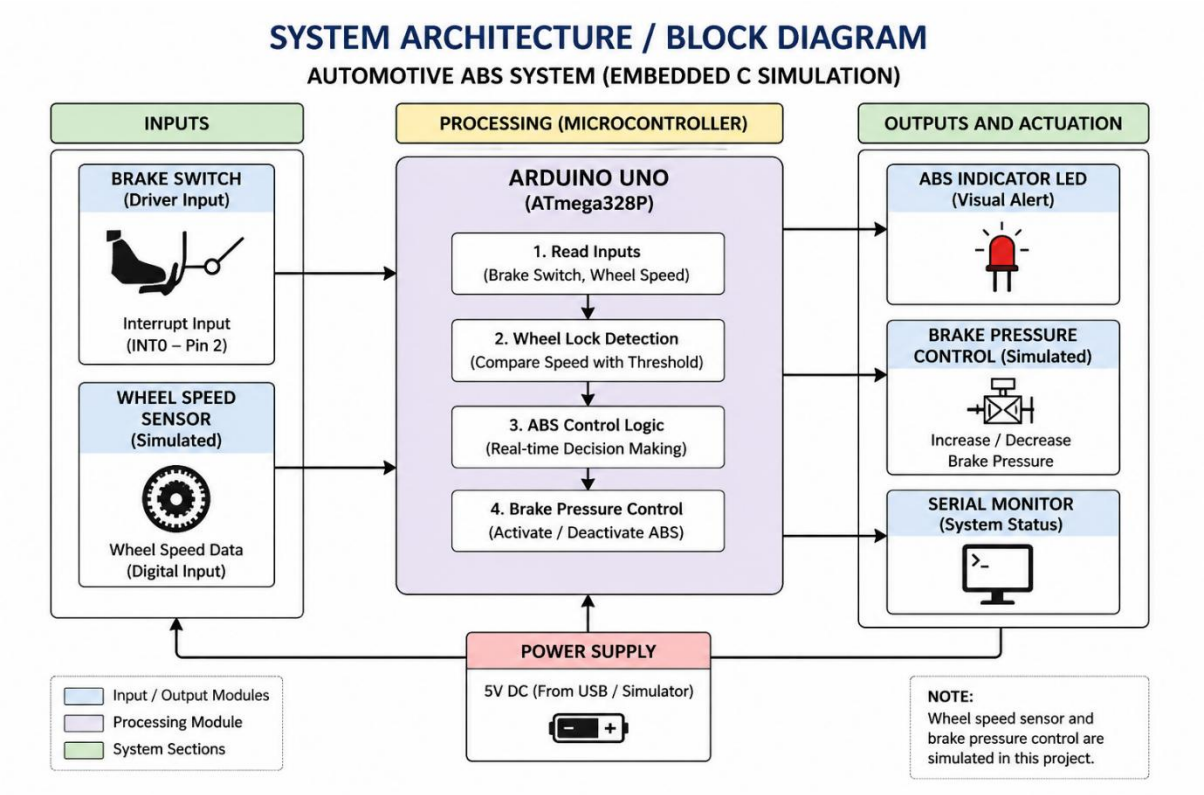
- Arduino Uno
- Push Button (Brake Switch)
- LED (ABS Indicator)

- 220 Ω Resistor
- Breadboard
- Jumper Wires
- Computer/Laptop

Software Requirements

- Wokwi Simulator
- Embedded C (Arduino IDE compatible)
- Web Browser
- GitHub
- PDF Editor (MS Word or similar)

System Architecture / Block Diagram



Description of Modules

Module	Description
Main Module	Initializes the system and coordinates all modules.
Sensor Module	Simulates wheel speed data and monitors wheel conditions.
Interrupt Module	Detects brake switch activation using an external interrupt.
ABS Controller Module	Determines wheel lock and activates or deactivates ABS.
Output Module	Controls the ABS indicator LED and displays system status on the Serial Monitor.

Algorithm

1. Start the system.
2. Initialize input and output pins.
3. Configure the external interrupt for the brake switch.
4. Initialize Serial communication.
5. Continuously monitor wheel speed.
6. Wait for the brake interrupt.
7. When the brake is pressed, read the current wheel speed.
8. Compare the wheel speed with the predefined threshold.
9. If the wheel speed is below the threshold:
    - Activate the ABS.

- Reduce simulated brake pressure.
- Turn ON the ABS indicator.

10. If the wheel speed returns to a safe value:
    - Deactivate the ABS.
    - Turn OFF the ABS indicator.
11. Repeat the monitoring process.

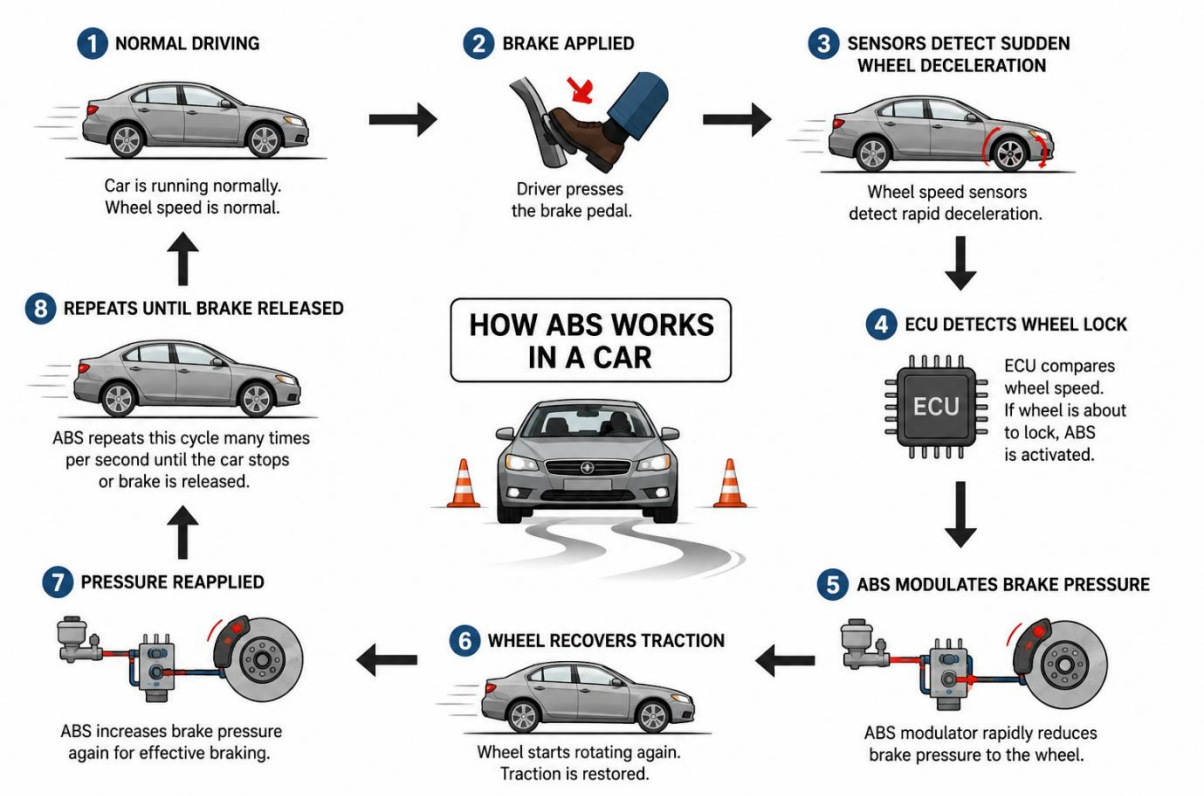
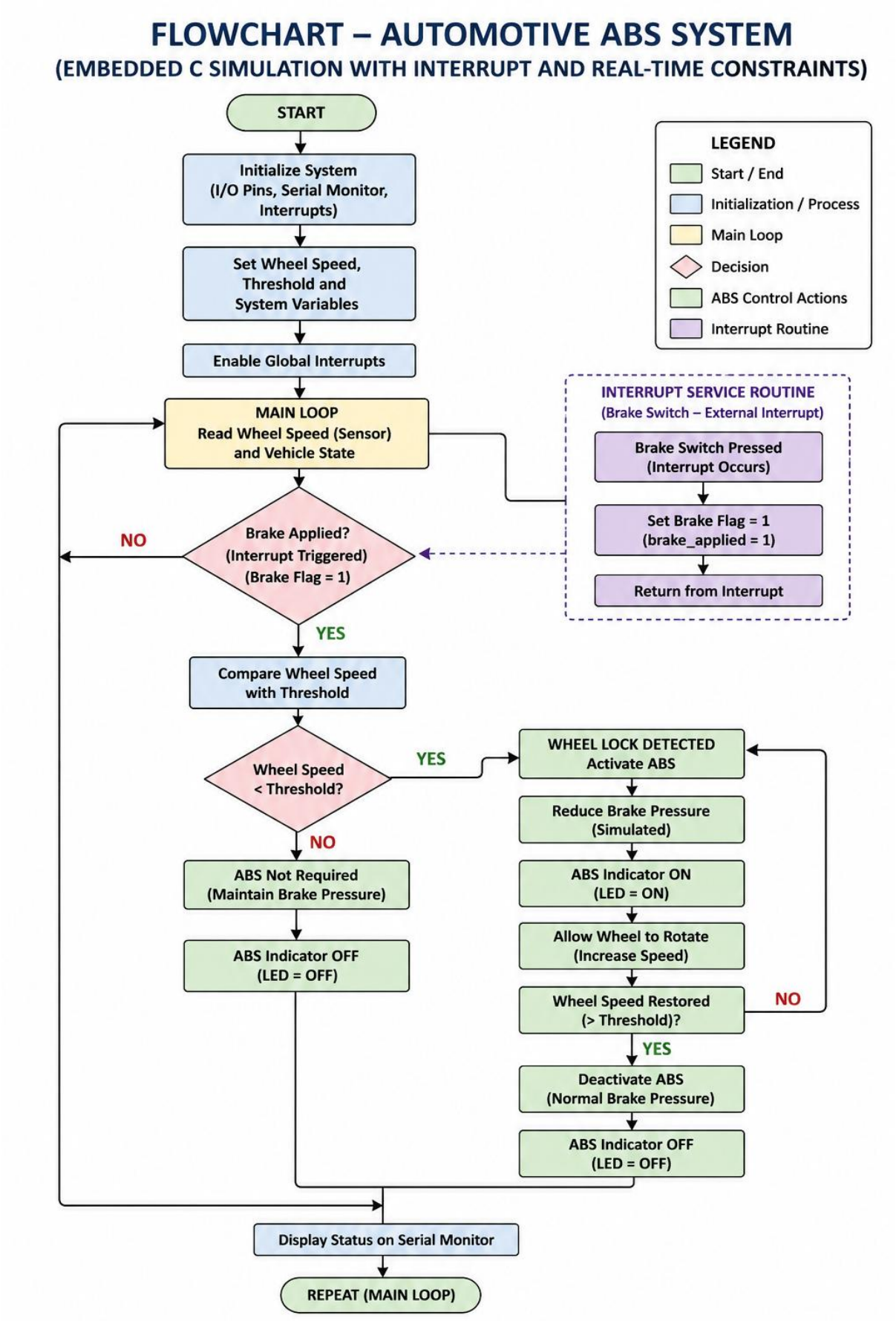


Figure 1: Working Principle of Automotive ABS System

Flowchart


FLOWCHART – AUTOMOTIVE ABS SYSTEM
(EMBEDDED C SIMULATION WITH INTERRUPT AND REAL-TIME CONSTRAINTS)
START
Initialize System (I/O Pins, Serial Monitor, Interrupts)
Set Wheel Speed, Threshold and System Variables
Enable Global Interrupts
MAIN LOOP
Read Wheel Speed (Sensor) and Vehicle State
Brake Applied? (Interrupt Triggered) (Brake Flag = 1)
NO
YES
Compare Wheel Speed with Threshold
Wheel Speed < Threshold?
YES
NO
ABS Not Required (Maintain Brake Pressure)
ABS Indicator OFF (LED = OFF)
WHEEL LOCK DETECTED Activate ABS
Reduce Brake Pressure (Simulated)
ABS Indicator ON (LED = ON)
Allow Wheel to Rotate (Increase Speed)
Wheel Speed Restored (> Threshold)?
NO
YES
Deactivate ABS (Normal Brake Pressure)
ABS Indicator OFF (LED = OFF)
Display Status on Serial Monitor
REPEAT (MAIN LOOP)
INTERRUPT SERVICE ROUTINE
(Brake Switch – External Interrupt)
Brake Switch Pressed (Interrupt Occurs)
Set Brake Flag = 1 (brake_applied = 1)
Return from Interrupt
LEGEND
Start / End
Initialization / Process
Main Loop
Decision
ABS Control Actions
Interrupt Routine

Embedded C/SystemC Implementation

1. sensor.h

```
#ifndef SENSOR_H

#define SENSOR_H

extern int wheelSpeed;

void updateWheelSpeed();

#endif
```

2. sensor.c

```
#include "sensor.h"

int wheelSpeed = 80;

void updateWheelSpeed()

{

   if (wheelSpeed > 0)

   {

      wheelSpeed -= 5;

   }

}
```

3. controller.h

```
#ifndef CONTROLLER_H

#define CONTROLLER_H

void ABS_Controller();
```

```
#endif
```

4. controller.c

```
#include <Arduino.h>
#include "sensor.h"
#include "controller.h"
#define LED_PIN 13
#define SPEED_THRESHOLD 20
extern volatile bool brakePressed;
void ABS_Controller()
{
   if (brakePressed)
   {
      if (wheelSpeed <= SPEED_THRESHOLD)
      {
         digitalWrite(LED_PIN, HIGH);
         Serial.println("================================");
         Serial.println("ABS ACTIVATED");
         Serial.print("Wheel Speed : ");
         Serial.println(wheelSpeed);
         Serial.println("Brake Pressure Reduced");
         Serial.println("================================");
```

```
            wheelSpeed += 15;
        }
        else
        {
            digitalWrite(LED_PIN, LOW);
            Serial.println("Brake Applied");
            Serial.print("Wheel Speed : ");
            Serial.println(wheelSpeed);
            Serial.println("ABS Not Required");
        }

        brakePressed = false;
    }
}
```

5. main.c (or main.ino in Wokwi)

```
#include <Arduino.h>
#include "sensor.h"
#include "controller.h"
#define BRAKE_PIN 2
#define LED_PIN 13
volatile bool brakePressed = false;
```

```
void brakeInterrupt()
{
  brakePressed = true;
}
void setup()
{
  pinMode(BRAKE_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
  attachInterrupt(
    digitalPinToInterrupt(BRAKE_PIN),
    brakeInterrupt,
    FALLING);
  Serial.println("--------------------------------");
  Serial.println("AUTOMOTIVE ABS SYSTEM");
  Serial.println("Embedded C Simulation");
  Serial.println("--------------------------------");
}
void loop()
{
  updateWheelSpeed();
```

```
    Serial.print("Current Wheel Speed : ");
    Serial.println(wheelSpeed);
    ABS_Controller();
    if (wheelSpeed <= 0)
    {
        wheelSpeed = 80;
        Serial.println();
        Serial.println("Vehicle Speed Reset");
        Serial.println();
    }
    delay(1000);
}
```

Expected Serial Monitor Output

```
--------------------------------
AUTOMOTIVE ABS SYSTEM
Embedded C Simulation
--------------------------------
Current Wheel Speed : 80
Current Wheel Speed : 75
Current Wheel Speed : 70
```

Brake Applied

Wheel Speed : 70

ABS Not Required

==========

Current Wheel Speed : 65

Current Wheel Speed : 60

Current Wheel Speed : 55

Current Wheel Speed : 50

Current Wheel Speed : 45

Current Wheel Speed : 40

Current Wheel Speed : 35

Current Wheel Speed : 30

Current Wheel Speed : 25

==========

ABS ACTIVATED

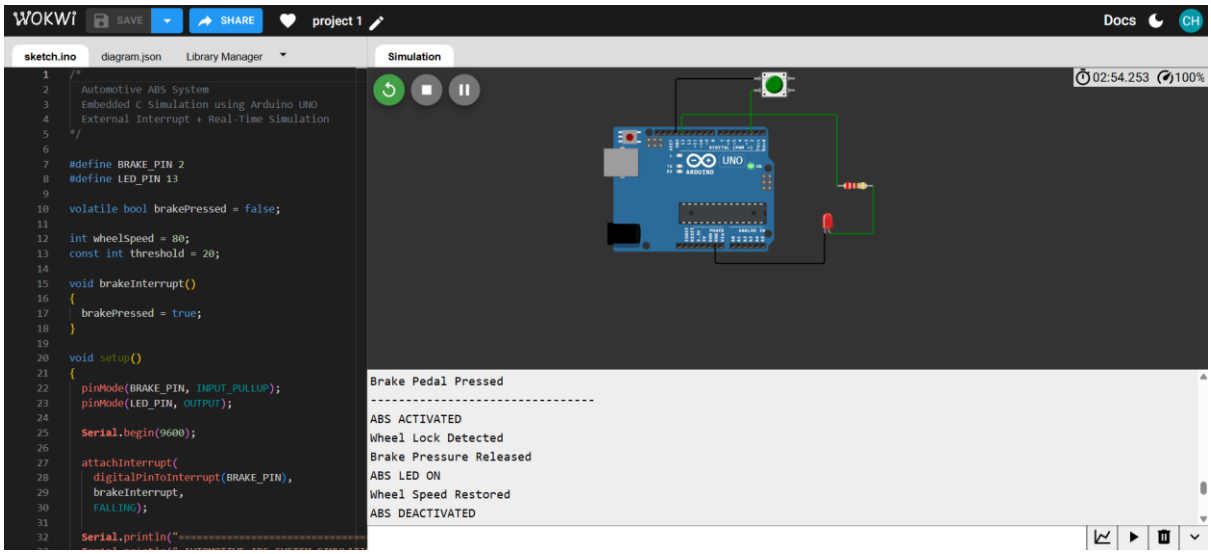
Wheel Speed : 20

Brake Pressure Reduced

==========

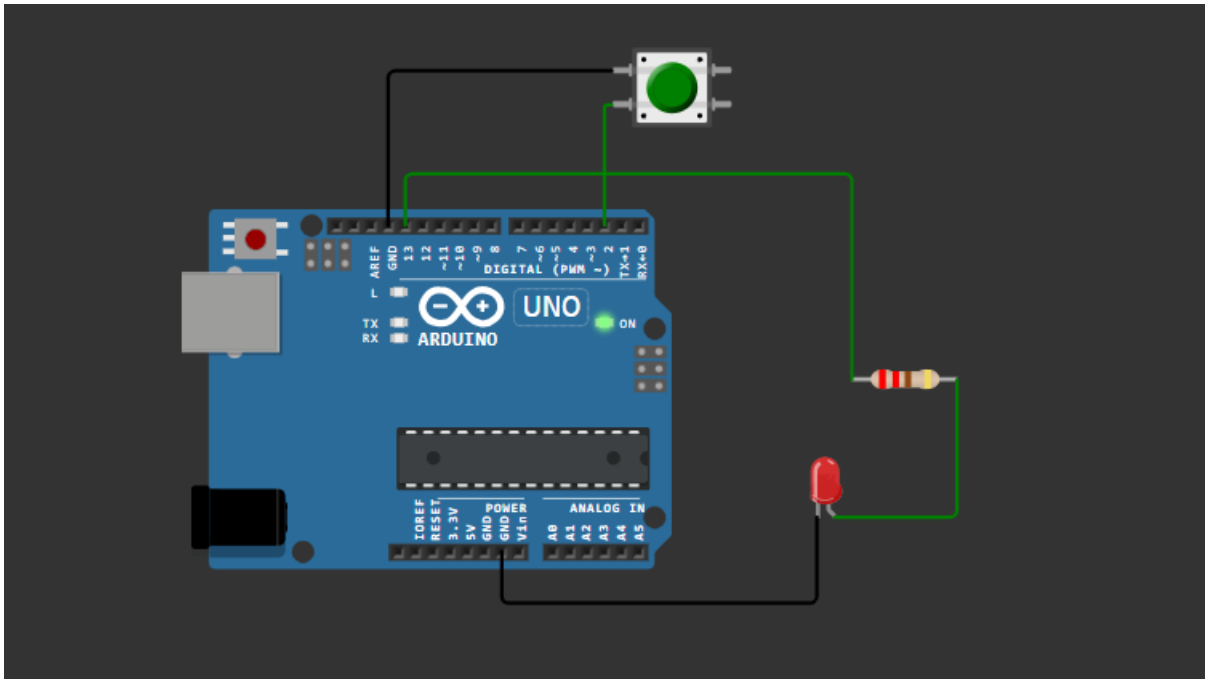
Current Wheel Speed : 35

Simulation Environment



Results and Screenshots

1. Circuit diagram



2. Simulation running with Serial Monitor

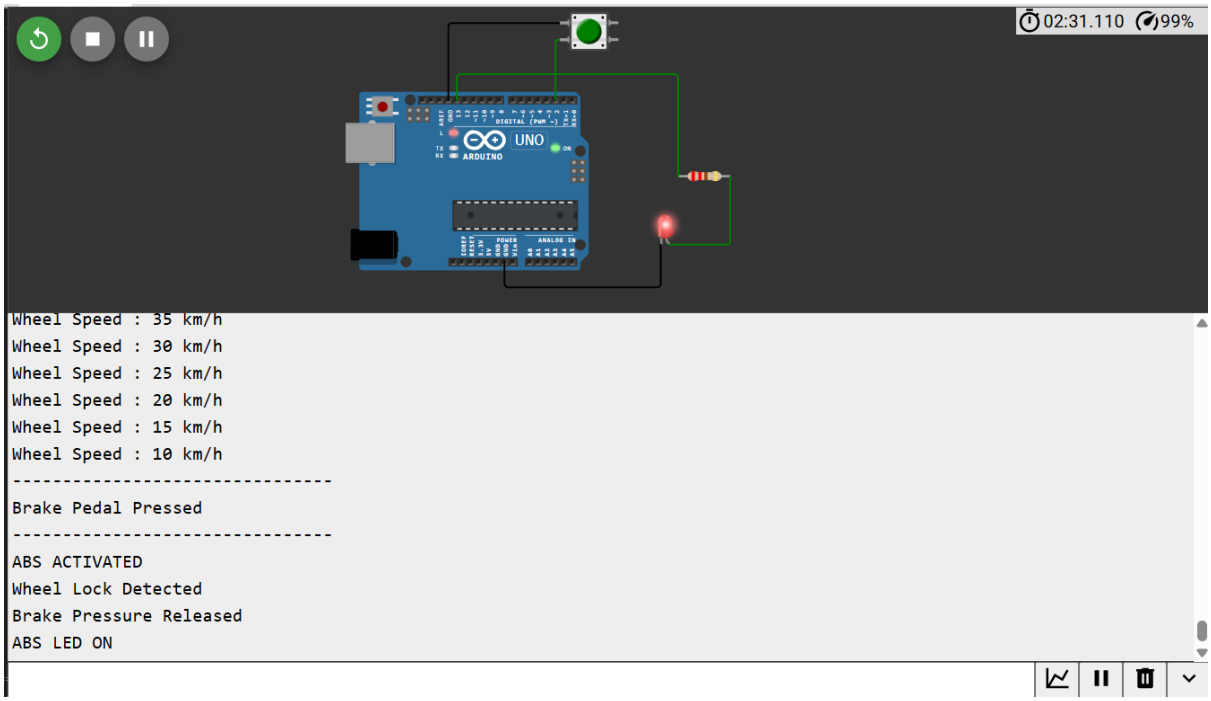
```
========================================
 AUTOMOTIVE ABS SYSTEM SIMULATION
========================================
System Started...

Wheel Speed : 80 km/h
Wheel Speed : 75 km/h
Wheel Speed : 70 km/h
Wheel Speed : 65 km/h
Wheel Speed : 60 km/h
Wheel Speed : 55 km/h
Wheel Speed : 50 km/h
Wheel Speed : 45 km/h
```

3. Brake button pressed

```
Wheel Speed : 80 km/h
Wheel Speed : 75 km/h
Wheel Speed : 70 km/h
Wheel Speed : 65 km/h
Wheel Speed : 60 km/h
--------------------------------
Brake Pedal Pressed
--------------------------------
Wheel Speed Normal
ABS NOT REQUIRED
--------------------------------
Wheel Speed : 55 km/h
Wheel Speed : 50 km/h
```

4. ABS activated



Test Cases

Test Case	Input Condition	Expected Output	Result
TC1	System Power ON	System initializes and displays startup message	Pass
TC2	Wheel speed above 20 km/h and brake pressed	ABS remains OFF	Pass
TC3	Wheel speed ≤ 20 km/h and brake pressed	ABS activates and LED turns ON	Pass
TC4	Wheel speed restored	ABS deactivates and LED turns OFF	Pass
TC5	Continuous operation	Wheel speed resets after reaching 0 km/h	Pass

Result Analysis

The Automotive ABS system was successfully simulated using Embedded C in the Wokwi environment. The external interrupt on Digital Pin 2 detected the brake switch immediately. The controller continuously monitored the simulated wheel speed. When the wheel speed fell below the threshold value of 20 km/h, the ABS logic activated, the LED turned ON, and brake pressure was simulated as released. Once the wheel speed recovered, the ABS deactivated automatically. The simulation confirmed that the system responded correctly under real-time conditions.

Conclusion

The Automotive Anti-lock Braking System (ABS) was successfully designed and implemented using Embedded C. The project demonstrated interrupt handling, wheel speed monitoring, and real-time control logic. The simulation correctly detected wheel lock conditions, activated the ABS mechanism, and restored normal braking operation. The implementation met the functional requirements and validated the effectiveness of interrupt-driven embedded programming for automotive safety applications.

GitHub Repository Link

https://github.com/gowthamch54/EmbeddedSystem_ECA1422.git